

UConn · School of Business

# Lab 3 — Scraper Detective

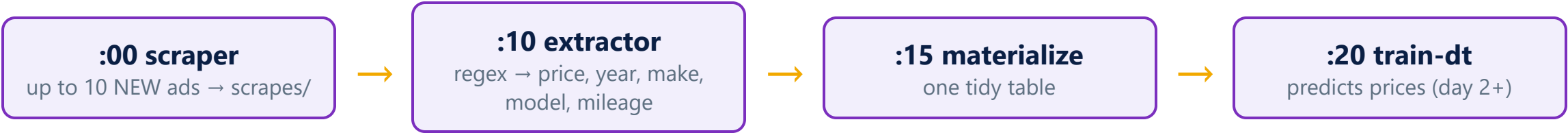
---

**Does your pipeline see what *you* see?**

OPIM 5512 · Module 3 · Solo · Colab · your own Google Cloud bucket

**Tonight you grade your robot — then fix it with one regular expression.**

# Your robot has been busy. Is it any good?



| Part                  | Time   | You do                                                                       |
|-----------------------|--------|------------------------------------------------------------------------------|
| 1 · Get it running    | 30 min | 5 green checks, then Force run scraper → extractor → materialize             |
| 2 · Search and see    | 25 min | your cars land in Colab's <b>Files panel</b> · search them · open 3 live ads |
| 3 · Grade the robot   | 30 min | a truth table for 5 cars → accuracy per field                                |
| 4 · Fix it three ways | 30 min | plain Python (given) · <b>your one-line regex</b> · a typo-fixer → re-grade  |

**Pipeline not ready? One switch: `USE_BACKUP_DATA = True`** loads Dr. Wanik's Hartford cars. Nobody sits out.

# Get it running (if it isn't yet)

- 1 • Final GCP Guide F26 (HuskyCT, Week 3.1), Steps 0–10 → **5 green checks** in your fork's Actions tab
- 2 • Cloud Scheduler: **tick the checkbox first**, then **Force run** — one at a time, wait for **Success**:  
① `craigslist-scraper-hourly` ② `extractor-per-listing-hourly` ③ `materialize-master-hourly`
- 3 • Cloud Storage → your bucket: `scrapes/` and `structured/` are there
- 4 • Already done? Skip to Part 2 and help a neighbor.

**Stuck? Ask Claude first:** paste the step, the command, and the FULL error. Then raise your hand.

# A map of your bucket

```
gs://your-project-id/
├─ scrapes/
│   └─ 20260928155246/           # one folder per scraper run (UTC)
│       ├── eH4UEPtCwKz2Vox1upEYCw.txt # the raw ad text
│       └─ index.csv             # which URL each file came from
├─ state/seen_post_ids.txt      # how it never grabs a car twice
└─ structured/
    ├── run_id=20260928155246/jsonl/ # the regex's answers, per ad
    ├── datasets/listings_master.csv  # ← tonight's table
    └─ preds/preds_master.csv        # predictions (once you have 2 days)
```

**Raw → structured → one table.** That's ETL. Tonight you check the T.

# Search and see

## IN YOUR NOTEBOOK

```
SEARCH_WORD = "toyota" # try sunroof, 4wd...

for i in range(len(cars)):
    row = cars.iloc[i]
    text = read_ad_text(row["source_txt"])
    if SEARCH_WORD.lower() in text.lower():
        matches.append(row)

3 of 30 ads mention 'toyota'
```

## FILES PANEL → MY\_CARS/01\_2013\_8500\_....TXT

```
2013 Hyunda Sonata GLS Limited for sale by owner
$8,500
(East Hartford)
image 1 of 13
2013
Hyunda Sonata GLS Limited
condition:
like new
odometer:
40,000
```

**Double-click any car on the left:** the .txt is the raw ad, the .json is what the robot extracted, and every car keeps its link to the live ad.

# Grade the robot: a truth table

Real Hartford cars. The robot nails the numbers. Make and model? **Vernon Rockville** is a town. **Hyunda** is the seller's typo:

| post_id           | price  | year   | make      | model         | mileage  |
|-------------------|--------|--------|-----------|---------------|----------|
| Sonata            | 8500 ✓ | 2013 ✓ | Hyunda ✗  | Sonata ✓      | 40000 ✓  |
| Accord            | 3800 ✓ | 2011 ✓ | Honda ✓   | Accord ✓      | 198000 ✓ |
| Elantra           | 4995 ✓ | 2013 ✓ | Contact ✗ | Information ✗ | 88000 ✓  |
| Odyssey           | 1800 ✓ | 2007 ✓ | Vernon ✗  | Rockville ✗   | 213000 ✓ |
| accuracy (5 cars) | 100%   | 100%   | 20%       | 40%           | 100%     |

**Truth-table rule: write the REAL make.** If the seller typed Hyunda, the truth is hyundai. Lowercase; model = first word.

## THE MYSTERY

# What is the make/model regex really grabbing?

### THE EXTRACTOR'S RULE (MODULE 3.2)

```
MAKE_MODEL_RE = re.compile(
    r"\b([A-Z][a-z]+)\s+([A-Z][A-Za-z0-9]+)")

# in words: the FIRST place where one
# Capitalized word is followed by another
```

### THE PAGE TITLE THE SELLER TYPED → WHAT IT GRABS

```
2011 Honda Accord LX ...      → Honda Accord ✓
2013 Hyunda Sonata GLS ...    → Hyunda Sonata (typo)
2013 hyundai elantra ...      → Contact Information
2007 Honda odyssey lx - Vernon Rockville → Vernon Rockville

# the reliable spot, near the photos:
'2007'
'Honda odyssey lx'
```

**Regex does exactly what you say, not what you mean.** Lowercase title? The first two Capitalized words are a button, or the town.

# Build it one piece at a time

| piece                                                      | means                           | piece                | means                                  |
|------------------------------------------------------------|---------------------------------|----------------------|----------------------------------------|
| <code>^</code> <code>\$</code> + <code>re.MULTILINE</code> | start / end of a <b>line</b>    | <code>\n</code>      | the line break                         |
| <code>\d{2}</code>                                         | exactly two digits              | <code>\s+</code>     | one or more spaces                     |
| <code>(?:19 20)</code>                                     | 19 or 20, grouped, not captured | <code>( ... )</code> | capture group → <code>.group(1)</code> |

step 1 a line that is only a year

step 2 ...followed by the next line

step 3 capture the first word (make)

step 4 a space, then capture the model

`r"^(?:19|20)\d{2}$"`

`r"^(?:19|20)\d{2}\n"`

`your turn`

`your turn`

→ matches '1980'

**Test fast at [regex101.com](https://regex101.com) (Python flavor).** Paste 20 lines of an ad as the test string and watch the match light up.

# Fix it three ways, then re-grade

WAY 1 GIVEN · WAY 2 IS YOURS · WAY 3 STRETCH

```
# Way 1: plain Python (given)
if len(line) == 4 and line.isdigit():
    words = next_line.lower().split()

# Way 2: the same idea in ONE regex line
BETTER_MAKE_MODEL_RE = re.compile(
    r"YOUR PATTERN HERE", re.MULTILINE)

# Way 3: fix typos with a list of makes
difflib.get_close_matches("hyunda", KNOWN_MAKES)
# ['hyundai']
```

THE NOTEBOOK RE-GRADES YOUR 5 CARS

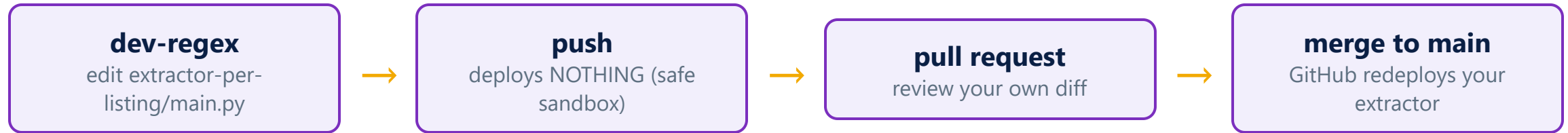
| field | robot | way 1 | way 2 | way 3 |
|-------|-------|-------|-------|-------|
| make  | 0.2   | 0.8   | ?     | 1.0   |
| model | 0.4   | 1.0   | ?     | 1.0   |

Dr. Wanik's Hartford sample, 5 cars

**Stretch:** run it on **all** your cars. How many ads does your pattern miss, and why?

**Same idea, three tools.** Rules are fast and cheap, and brittle. In A07, Gemini tries to beat way 3.

# Ship your regex: **branch** → **PR** → **deploy**



- Only **main** deploys: the workflows run on main, and Google Cloud only trusts main.
- Keep your own analysis in **separate files** (notebooks) so **Sync fork** still works when Dr. Wanik updates the course code.

**That's CI/CD:** code change → review → automatic deploy. No clicking around in the cloud.

# Good advice

- **Colab pop-up:** pick the Gmail you used for **Google Cloud**, not necessarily your UConn account.
- **listings\_master.csv not found?** Force run the extractor, then materialize-master; wait for **Success**.
- **Your regex matches nothing?** Print the lines with `repr()`: hidden spaces and line breaks show up.
- **Only 10 cars?** That's one scrape. It adds up to 10 **new** cars every hour; leave it running.
- **Pipeline not ready?** `USE_BACKUP_DATA = True` loads Dr. Wanik's Hartford cars. Do the lab now, fix your pipeline after.

## DEFINITION OF DONE

# Submit Lab 3 on HuskyCT

- ☐ Part 2: search results + 2–3 sentences on what the raw text keeps and loses
- ☐ Part 3: your **truth table** for 5 cars + the **before** accuracy table
- ☐ Part 4: your **BETTER\_MAKE\_MODEL\_RE** line + the **before vs after** table
- ☐ Part 5: truth table **saved for A07** (Gemini gets graded on the same 5 cars) + three short answers
- ☐ File → Download → **.ipynb** → **Submit Lab 3** (due **Thu Oct 8, 11:59 PM**)

**Leave the robot running.** Next: A06 (update the ETL), then A07, where Gemini reads the ads instead of a regex.